

Проект 2. Библиотека Pandas

В этом проекте ты познакомишься с библиотекой Pandas, которая является мощным инструментом для анализа и обработки данных в Python. Ты научишься создавать DataFrame из разных источников, фильтровать и агрегировать данные.

Содержание

- [Глава 1](#)
 - [Pandas и его концепции](#)
 - [Создание DataFrame](#)
 - [Основные способы доступа к данным](#)
 - [Условия фильтрации](#)
 - [Сортировка данных](#)
 - [Агрегация данных](#)
 - [Группировка данных](#)
- [Глава 2](#)
 - [Практические задания](#)
 - [Задание 1. Сортировка фильмов по рейтингу](#)
 - [Задание 2. Агрегация данных по режиссерам](#)
 - [Задание 3. Фильтрация данных по жанру](#)

Глава 1

Pandas и его концепции

Для начала работы в папке src скачай ноутбук project_6.ipynb — это твоя рабочая тетрадь на проект, для удобства в ней можно работать и запускать код.

Представь, что ты работаешь аналитиком данных в интернет-магазине. Тебе нужно проанализировать много данных о продажах за последний квартал, чтобы понять, какие товары продаются лучше всего. В этом поможет Pandas — библиотека Python для анализа и обработки данных, она позволяет быстро и эффективно анализировать, фильтровать и преобразовывать данные.

Ты можешь подумать, что то же самое можно сделать в Excel, но нет. Вот какие плюсы есть у Pandas:

1. Pandas может обрабатывать гораздо большие объемы данных, тогда как в Excel максимальное количество строк — 1 048 576.
2. Pandas позволяет писать скрипты, которые будут выполнять сложные операции с данными.
3. Pandas оптимизирован для работы с большими наборами данных и может выполнять операции гораздо быстрее.

Существует две основные концепции Pandas:

1. **Series** — это одномерный массив, который может содержать данные любого типа. Это основа для создания DataFrame.

2. **DataFrame** — это основная структура данных в Pandas. Это двумерная таблица, похожая на электронную таблицу, где строки и столбцы имеют имена. Именно с такими таблицами тебе предстоит работать.

На таблице ниже ты можешь увидеть, что такое Series и DataFrame.

Series 1		Series 2		Series 3		DataFrame			
Orange		Apple		Banana		Orange	Apple	Banana	
0	4	0	4	0	4	0	4	4	4
1	5	1	5	1	5	1	5	5	5
2	6	2	6	2	6	2	6	6	6
3	3	3	3	3	3	3	3	3	3
4	1	4	1	4	1	4	1	1	1

Создание DataFrame

Создать DataFrame можно из разных источников:

- 1. Создание DataFrame из списка.

Это один из самых простых и распространенных способов работы с данными в Pandas. Давай разберем, как это сделать на примере.

Например, у тебя есть информация о людях, эта информация выглядит как обычные списки, но хочется представить ее в удобной форме, чтобы было легче анализировать. Как раз Pandas поможет превратить этот список в таблицу, чтобы дальше ее можно было анализировать, сортировать, искать закономерности и многое другое.

```
# Импортируем библиотеку Pandas:
import pandas as pd

# Создаем список данных:
data = [
    ["Alice", 25],
    ["Bob", 30],
    ["Charlie", 35],
    ["Diana", 28]
]

# Преобразуем список в DataFrame и задаем заголовки столбцов:
df = pd.DataFrame(data, columns=["Name", "Age"])

# Выведем новую таблицу:
print(df)

# Вывод:
# | № | Name      | Age |
```

```
# | --- | ----- | ----- |
# | 0 | Alice | 25 |
# | 1 | Bob   | 30 |
# | 2 | Charlie | 35 |
# | 3 | Diana | 28 |
```

2. Создание DataFrame из словаря.

Иногда у тебя есть данные, представленные в виде обычного словаря Python, где ключи — это названия столбцов, а значения — сами данные. Давай разберемся, как превратить словарь в удобную таблицу.

```
# У тебя есть такая информация о студентах:
# Ключ "Имя" соответствует именам студентов.
# Ключ "Возраст" хранит их возраст.
# Ключ "Оценка" содержит оценки студентов.

students_dict = {
    "Имя": ["Анна", "Борис", "Виктор"],
    "Возраст": [20, 22, 21],
    "Оценка": [90, 85, 95]
}

# Импортируем библиотеку Pandas:
import pandas as pd

# Преобразуем словарь в DataFrame:
students_df = pd.DataFrame(students_dict)

# Выведем новую таблицу:
print(students_df)
```

```
#Вывод:
# | № | Имя      | Возраст | Оценка |
# | --- | ----- | ----- | ----- |
# | 0 | Анна    | 20      | 90      |
# | 1 | Борис   | 22      | 85      |
# | 2 | Виктор  | 21      | 95      |
```

3. Создание DataFrame из файла (CSV).

Очень часто данные хранятся в файлах CSV. Это текстовые файлы, где каждая строка представляет собой отдельный ряд данных, а столбцы отделяются запятыми.

```
# Файл содержит информацию:
# Имя, Возраст, Оценка
# Анна, 20, 90
# Борис, 22, 85
# Виктор, 21, 95
```

```
# Импортируем библиотеку Pandas:
import pandas as pd

# Читаем CSV-файл и создаем DataFrame.
# Важно! Обязательно укажи путь к файлу, где он лежит, в ином случае код не
запустится.
csv_df = pd.read_csv('students.csv')

# Выведем новую таблицу:
print(csv_df)

#Вывод:
# | № | Имя      | Возраст | Оценка |
# | ---|-----|-----|-----|
# | 0 | Анна    | 20      | 90      |
# | 1 | Борис   | 22      | 85      |
# | 2 | Виктор  | 21      | 95      |
```

Основные способы доступа к данным

Чтобы взять нужные данные, нужно знать адрес ячейки, строки или столбца. Давай разберем 4 основных способа доступа к данным на конкретном примере. Представим, что есть таблица DataFrame со студентами:

Индекс	Имя	Возраст	Оценка
0	Анна	20	90
1	Борис	22	85
2	Виктор	21	95

1. Получение отдельного столбца. Чаще всего нужно смотреть данные только по одному признаку (например, только имена):

```
# Через квадратные скобки:
a = students_df['Имя']

# Через точку:
b = students_df.Имя

# Если нужно выбрать несколько столбцов, передаем список столбцов (двойные
скобки):
c = students_df[['Имя', 'Возраст']]

# Посмотреть результаты:
print(a,b,c)
```

2. Выбор строк. В Pandas есть два «помощника» для выбора строк:

Помощник	Что означает	Как считает	Пример
.loc	Location (По метке)	Смотрит на имя индекса	df.loc[0] (найдет строку с индексом 0)
.iloc	Integer Loc (По позиции)	Смотрит на порядковый номер (как в списке)	df.iloc[0] (найдет самую первую строку)

```
# Получить строку с индексом 0 (Анна):
first_row = students_df.loc[0]

# Получить первую строку по счету (тоже Анна):
second_row = students_df.iloc[0]

# .loc включает ПОСЛЕДнюю цифру диапазона!
# Вернет строки с индексами 0, 1 и 2 (включительно):
subset_rows = students_df.loc[:2]

# .iloc работает как обычный Python (последнюю цифру НЕ включает).
# Вернет строки под номерами 0, 1 и 2 (диапазон до 3):
subset_rows_by_position = students_df.iloc[:3]
```

3. Выбор столбцов. Иногда нужно взять не все столбцы, а только часть:

```
# Взять все строки (:), но столбцы от начала до 'Возраст' включительно:
subset_columns_loc = students_df.loc[:, : 'Возраст']

# Взять все строки (:), но только первые 2 столбца (0 и 1).
# Столбец под номером 2 (Оценка) не войдет.
subset_columns_iloc = students_df.iloc[:, :2]
```

4. Обращение к отдельной ячейке. Если нужно узнать конкретное значение (например, возраст Анны):

```
# [Строка с индексом 0, Столбец 'Возраст']
value = students_df.loc[0, 'Возраст'] # Результат: 20

# [Строка номер 2 (Виктор), Столбец номер 1 (Возраст)]
# Нумерация начинается с 0!
value = students_df.iloc[2, 1] # Результат: 21
```

Условия фильтрации

Фильтрация данных — это операция, позволяющая оставить в таблице только те строки, которые соответствуют указанным условиям.

Например, ты можешь захотеть отобрать всех сотрудников, чей возраст превышает определенное значение, или товары, цена которых меньше указанного уровня.

Какие бывают условия фильтрации

Оператор	Описание	Пример условия
\$<\$	меньше	df['Возраст'] < 30
\$>\$	больше	df['Зарплата'] > 50000
\$==\$	равно	df['Имя'] == 'Анна'
\$!=\$	не равно	df['Статус'] != 'Уволен'
\$<=\$	меньше или равно	df['Оценка'] <= 90
\$>=\$	больше или равно	df['Возраст'] >= 18

Как поставить фильтр на несколько условий

Чаще всего возникает необходимость комбинировать несколько условий. Тут пригодятся логические операторы:

Оператор	Название	Описание
&	И (AND)	Используется, когда оба условия должны соблюдаться
\$	\$	ИЛИ (OR)
~	НЕ (NOT)	Поможет отбросить ненужные строки (инверсия условия)

Допустим, есть таблица сотрудников, состоящая из трех столбцов: имя сотрудника, его возраст и зарплата.

Имя	Возраст	Зарплата
Анна	25	50000
Борис	30	60000
Виктор	35	70000
Елена	28	55000

С таблицей нужно выполнить 3 задачи:

- Отбираем молодых сотрудников (возраст менее 30 лет).
- Ищем высокооплачиваемых работников (зарплата выше 55 тыс. руб.).
- Отбираем сотрудников с опытом работы от 25 до 35 лет.

```
import pandas as pd

# Таблица сотрудников:
employees = pd.DataFrame({
```

```

    'Имя': ['Анна', 'Борис', 'Виктор', 'Елена'],
    'Возраст': [25, 30, 35, 28],
    'Зарплата': [50000, 60000, 70000, 55000]
})

# 1. Молодые сотрудники (< 30 лет):
young_employees = employees[employees['Возраст'] < 30]

# 2. Высокооплачиваемые сотрудники (> 55 тыс. руб.):
high_paid_employees = employees[employees['Зарплата'] > 55000]

# 3. Опытные сотрудники (от 25 до 35 лет):
experienced_employees = employees[(employees['Возраст'] >= 25) &
    (employees['Возраст'] <= 35)]

# Посмотреть результаты:
print(young_employees)
print(high_paid_employees)
print(experienced_employees)

# Вывод.
# Молодые сотрудники:
# | Имя | Возраст | Зарплата |
# |-----|-----|-----|
# | Анна | 25 | 50000 |
# | Елена | 28 | 55000 |

# Высокооплачиваемые сотрудники:
# | Имя | Возраст | Зарплата |
# |-----|-----|-----|
# | Виктор | 35 | 70000 |

# Опытные сотрудники:
# | Имя | Возраст | Зарплата |
# |-----|-----|-----|
# | Борис | 30 | 60000 |
# | Виктор | 35 | 70000 |
# | Елена | 28 | 55000 |

```

Сортировка данных

Сортировка данных — это операция, которая упорядочивает строки в DataFrame по указанному столбцу или нескольким столбцам.

Эта процедура полезна, когда нужно расположить данные в нужном порядке для последующего анализа или вывода.

Способы сортировки:

- Сортировка по возрастанию (ascending order): строки располагаются от меньших значений к большим.

- Сортировка по убыванию (descending order): строки располагаются от больших значений к меньшим.

```
import pandas as pd

# Исходные данные:
# | Имя | Возраст | Зарплата |
# |----|:-----:|-----:|
# | Анна | 25 | 50000 |
# | Борис | 30 | 60000 |
# | Виктор | 35 | 70000 |
# | Елена | 28 | 55000 |

# Сортировка по возрасту по возрастанию:
sorted_employees = employees.sort_values(by='Возраст')

# Посмотреть результаты:
print(sorted_employees)

# Вывод:
# | № | Имя | Возраст | Зарплата |
# |--:|:-----|-----:|-----:|
# | 0 | Анна | 25 | 50000 |
# | 3 | Елена | 28 | 55000 |
# | 1 | Борис | 30 | 60000 |
# | 2 | Виктор | 35 | 70000 |

# Сортировка по зарплате по убыванию:
sorted_employees = employees.sort_values(by='Зарплата', ascending=False)

# Посмотреть результаты:
print(sorted_employees)

# Вывод:
# | № | Имя | Возраст | Зарплата |
# |--:|:-----|-----:|-----:|
# | 2 | Виктор | 35 | 70000 |
# | 1 | Борис | 30 | 60000 |
# | 3 | Елена | 28 | 55000 |
# | 0 | Анна | 25 | 50000 |
```

Агрегация данных

Агрегация — это процесс сбора и расчета общих характеристик данных, таких как среднее значение, сумма, минимум, максимум и прочее.

Это ключевой этап при анализе данных, позволяя получать общие представления о распределении данных.

Наиболее частые операции агрегации:

- Среднее значение (mean()) — среднее арифметическое по данному столбцу.
- Сумма (sum()) — суммарное значение по столбцу.
- Минимум (min()) — наименьшее значение среди данных.
- Максимум (max()) — наибольшее значение среди данных.
- Стандартное отклонение (std()) — мера разброса данных относительно среднего значения.

```
import pandas as pd

# Исходные данные:
# | Имя | Возраст | Зарплата |
# |----|:-----|:-----:|
# | Анна | 25 | 50000 |
# | Борис | 30 | 60000 |
# | Виктор | 35 | 70000 |
# | Елена | 28 | 55000 |

# Рассчитаем среднюю зарплату сотрудников:
average_salary = employees['Зарплата'].mean()

# Посмотреть результаты:
print(average_salary)

# Вывод:
# 58750.0
```

Группировка данных

Группировка — это операция разделения данных на подгруппы по какому-либо признаку (категориям) с последующим расчетом определенных показателей для каждой подгруппы отдельно. Особенно полезна при сравнении категориальных данных.

Часто путают агрегацию и группировку, но все-таки это разное.

Характеристика	Агрегация (.mean())	Группировка (.groupby().mean())
Что делает	Считает одно число по всей таблице	Считает по одному числу для каждой группы
Результат	Одно значение	Таблица с количеством строк = количеству групп
Пример	«Средняя зарплата: 58 750 ₽»	«IT: 61 667 ₽, HR: 57 667 ₽»
Код	df['Зарплата'].mean()	df.groupby('Отдел')['Зарплата'].mean()

```
import pandas as pd

# Исходные данные:
df = pd.DataFrame({
    'Имя': ['Анна', 'Борис', 'Виктор', 'Елена', 'Дмитрий', 'Ольга'],
```

```
'Отдел': ['IT', 'HR', 'IT', 'HR', 'IT', 'HR'],
'Возраст': [25, 30, 35, 28, 32, 29],
'Зарплата': [50000, 60000, 70000, 55000, 65000, 58000]
})

# Смотрим среднюю зарплату по отделам:
avg_salary_by_dept = df.groupby('Отдел')['Зарплата'].mean()

# Посмотреть результаты:
print(avg_salary_by_dept)

# Вывод:
# Отдел
# HR      57666.666667
# IT      61666.666667
```

Глава 2

Практические задания

Задание 1. Сортировка фильмов по рейтингу

1. Загрузи данные kinopoisk-top250.csv.
2. Создай DataFrame с фильмами из файла.
3. При необходимости отсортируй по столбцу rating_ball в порядке убывания.
4. Выбери топ-5 фильмов.
5. Выведи результат на экран.
6. Сохрани файл project_6_task_1.ipynb и загрузи его в папку src.

Задание 2. Агрегация данных по режиссерам

1. Сделай группировку фильмов по столбцу director.
2. Вычисли среднее значение столбца rating_ball для каждого режиссера.
3. Отсортируй результат по среднему рейтингу в порядке убывания.
4. Выведи результат на экран.
5. Сохрани файл project_6_task_2.ipynb и загрузи его в папку src.

Задание 3. Фильтрация данных по жанру

1. Примени настройки фильтрации:
 - год выпуска ≥ 1990 ;
 - рейтинг rating_ball выше 8.5;
 - слово «жены» в столбце overview.
2. Объедини все условия.
3. Примени условия к DataFrame для выбора подходящих фильмов.
4. Выведи результат на экран.
5. Сохрани файл project_6_task_3.ipynb и загрузи его в папку src.3